ECE 8908/ENGI 9808 – Applied Machine Learning & Neural Networks

Lab 1 Instructions

Due Date: Saturday, October 17, 2026 at 11:59 pm

Introduction

A. Classification

As the world continues to make huge strides in intelligent systems, it is expected that a number of restaurants will be equipped with robots that will assist in service delivery and the daily running of restaurants. Suppose that in one restaurant, the robot helps the staff organize utensils. In this case, the robot encounters a variety of kitchen tools: spoons, forks, and knives all mixed together. To assist efficiently, the robot needs to recognize and sort these tools automatically.

Assume the robot's camera only captures black-and-white images, so it cannot rely on color to tell the objects apart. Instead, it must learn to identify the shapes, edges, and patterns that make a spoon different from a dinner fork or a dinner knife.

Your task in this lab is to step into the role of the robot's programmer. You will build a machine learning classifier that can recognize these black-and-white images and correctly categorize each object. In this exercise, you will explore how to:

- Prepare the dataset
- Train and validate your classifier
- Evaluate its performance using key metrics

By the end of the lab, your model should be able to help the smart kitchen robot sort utensils, giving it the ability to understand its environment just like a human would.

The dataset for this problem can be found in the "Utensil images". The "SVC.py" file has the code that implements the instructions listed below.

Lab Instructions

- Python version: 3.8
- Libraries: glob, numpy, sklearn, skimage, pathlib.

1. **Load the Dataset**
    - Load the "Utensil images" folder from your directory and examine its contents.
    - How many samples are in the dataset?

2. **Split the Dataset**
    - Split the dataset into training and test sets using the random train-test split from the "sklearn" library: 70% training, 15% validation and 15% testing.
    - How many training and test samples are in the training and test sets?
3. **Select and Configure the Algorithm**
    - Use the Support Vector Classifier (SVC) as the classification algorithm.
4. **Hyperparameter Exploration**
    - Train the SVC with the combinations of hyperparameters provided in Table 1.
    - For each combination, record the training and validation accuracies in a table
    - Provide comments on model behavior, noting instances of overfitting, underfitting, or good generalization (**NB:** For this lab, a good performing model should achieve at least 90% accuracy)
    - Report your results and observations following the template of Table 1.

Table 1: Performance of the SVC using different combinations of hyperparameters

Hyperparameters	**Training accuracy**	**Val. accuracy**	**Comments (overfitting/underfitting/good generalization)**
kernel='linear', C=0.1, max_iter=-1, coef0=0.0, tol=1e-3, degree=3			
kernel='linear', C=1.0, max_iter=-1, coef0=0.0, tol=1e-4, degree=3			
kernel='linear', C=10.0, max_iter=-1, coef0=0.0, tol=1e-3, degree=3			
kernel='linear', C=0.5, max_iter=-1, coef0=0.0, tol=1e-4, degree=3			
kernel='linear', C=5.0, max_iter=-1, coef0=0.0, tol=1e-2, degree=3			
kernel='linear', C=1.0, max_iter=5000, coef0=0.0, tol=1e-3, degree=3			
kernel='linear', C=0.1, max_iter=10000, coef0=0.0, tol=1e-3, degree=3			
kernel='linear', C=2.0, max_iter=2000, coef0=0.0, tol=1e-4, degree=3			
kernel='linear', C=0.5, max_iter=-1, coef0=0.0, tol=1e-2, degree=3			
kernel='linear', C=3.0, max_iter=-1, coef0=0.0, tol=1e-3, degree=3			
kernel='linear', C=1.0, max_iter=1000, coef0=0.0, tol=1e-2, degree=3			
kernel='linear', C=0.05, max_iter=-1, coef0=0.0, tol=1e-4, degree=3			

kernel='sigmoid', C=0.1, gamma=0.01, coef0=0.0, tol=1e-3, max_iter=-1			
kernel='sigmoid', C=1.0, gamma=0.01, coef0=0.0, tol=1e-3, max_iter=-1			
kernel='sigmoid', C=10.0, gamma=0.01, coef0=0.0, tol=1e-3, max_iter=-1			
kernel='sigmoid', C=0.5, gamma=0.05, coef0=0.0, tol=1e-3, max_iter=-1			
kernel='sigmoid', C=1.0, gamma=0.05, coef0=0.0, tol=1e-4, max_iter=-1			
kernel='sigmoid', C=5.0, gamma=0.05, coef0=0.1, tol=1e-3, max_iter=-1			
kernel='sigmoid', C=1.0, gamma=0.1, coef0=0.0, tol=1e-3, max_iter=5000			
kernel='sigmoid', C=0.1, gamma=0.1, coef0=0.1, tol=1e-4, max_iter=10000			
kernel='sigmoid', C=2.0, gamma=0.1, coef0=0.0, tol=1e-3, max_iter=2000			
kernel='sigmoid', C=0.5, gamma=0.2, coef0=0.1, tol=1e-2, max_iter=-1			
kernel='sigmoid', C=3.0, gamma=0.2, coef0=0.0, tol=1e-3, max_iter=-1			
kernel='sigmoid', C=1.0, gamma=0.05, coef0=0.05, tol=1e-4, max_iter=1000			
kernel='poly', C=0.1, degree=2, gamma=0.01, coef0=0.0, tol=1e-3, max_iter=-1			
kernel='poly', C=0.5, degree=2, gamma=0.05, coef0=0.0, tol=1e-3, max_iter=-1			
kernel='poly', C=1.0, degree=2, gamma=0.1, coef0=0.0, tol=1e-3, max_iter=-1			
kernel='poly', C=5.0, degree=3, gamma=0.01, coef0=0.1, tol=1e-4, max_iter=-1			

kernel='poly', C=10.0, degree=3, gamma=0.05, coef0=0.1, tol=1e-3, max_iter=-1			
kernel='poly', C=1.0, degree=3, gamma=0.1, coef0=0.0, tol=1e-4, max_iter=5000			
kernel='poly', C=0.1, degree=3, gamma=0.1, coef0=0.05, tol=1e-3, max_iter=10000			
kernel='poly', C=2.0, degree=4, gamma=0.01, coef0=0.0, tol=1e-3, max_iter=2000			
kernel='poly', C=0.5, degree=4, gamma=0.05, coef0=0.1, tol=1e-2, max_iter=-1			
kernel='poly', C=3.0, degree=4, gamma=0.1, coef0=0.05, tol=1e-3, max_iter=-1			
kernel='poly', C=1.0, degree=5, gamma=0.05, coef0=0.0, tol=1e-4, max_iter=1000			
kernel='poly', C=5.0, degree=5, gamma=0.1, coef0=0.1, tol=1e-3, max_iter=-1			
kernel='poly', C=10.0, degree=5, gamma=0.2, coef0=0.0, tol=1e-3, max_iter=-1			
kernel='rbf', C=0.1, gamma=0.01, tol=1e-3, max_iter=-1, coef0=0.0, degree=3			
kernel='rbf', C=0.5, gamma=0.01, tol=1e-4, max_iter=-1, coef0=0.0, degree=3			
kernel='rbf', C=1.0, gamma=0.01, tol=1e-3, max_iter=-1, coef0=0.0, degree=3			
kernel='rbf', C=5.0, gamma=0.01, tol=1e-3, max_iter=-1, coef0=0.0, degree=3			
kernel='rbf', C=10.0, gamma=0.01, tol=1e-4, max_iter=-1, coef0=0.0, degree=3			
kernel='rbf', C=1.0, gamma=0.05, tol=1e-3, max_iter=5000, coef0=0.0, degree=3			
kernel='rbf', C=0.1, gamma=0.05, tol=1e-4, max_iter=10000, coef0=0.0, degree=3			

kernel='rbf', C=2.0, gamma=0.05, tol=1e-3, max_iter=2000, coef0=0.0, degree=3			
kernel='rbf', C=0.5, gamma=0.1, tol=1e-2, max_iter=-1, coef0=0.0, degree=3			
kernel='rbf', C=3.0, gamma=0.1, tol=1e-3, max_iter=-1, coef0=0.0, degree=3			
kernel='rbf', C=1.0, gamma=0.2, tol=1e-4, max_iter=1000, coef0=0.0, degree=3			
kernel='rbf', C=5.0, gamma=0.2, tol=1e-3, max_iter=-1, coef0=0.0, degree=3			
kernel='rbf', C=10.0, gamma=0.2, tol=1e-3, max_iter=-1, coef0=0.0, degree=3			

5. **Cross-Validation Analysis**
    - Repeat steps 4 and 5 using cross-validation techniques:
        - K-fold for 5, 10 and 15 folds (3 tables)
        - Repeated K-fold for 5, 10 and 15 folds with 5 and 10 repetitions each (6 tables)
6. **Hyperparameter selection**
    - Identify the best-performing models based on your hyperparameter investigation for the random split, K-fold and repeated K-fold cross-validation.
7. **Compare and Conclude**
    - Evaluate the accuracy of the model on the test data. Here, the training and validation data are combined into one general dataset for training the classifier one final time.
    - Obtain the confusion matrices, precision, recall and F1-scores of the classifier based on the 3 techniques. If the same set of hyperparameters produces the best performance in all 3 cases, you may report the result once for all.
    - Discuss the results.

List of deliverables:

- Video (webm, mp4 (most preferred), mov)
    - Maximum of 10 minutes in length
    - Not larger than 300 MB
    - Your webcam must be on for this recording. This is mandatory.

- Lab report (pdf file)
  Note: Each file should be named using this template: "student number_clf_lab1.file type"

Regression

With the rapid growth of online marketplaces, buying and selling of used cars have become increasingly data-driven. Unlike new vehicles, where pricing is standardized, the value of a used car is influenced by multiple interrelated factors such as year of manufacture, mileage, engine capacity, fuel type, maximum power, and number of prior owners. These factors often interact in complex ways, making accurate price estimation a non-trivial task.

Accurate price prediction is important for both consumers and dealerships. For buyers, it ensures that they are paying a fair price, while for sellers, it helps set a competitive price that maximizes profit without deterring potential customers. Traditional rule-of-thumb approaches are often subjective and fail to account for the large variability in car characteristics and market demand. However, using machine learning, one can develop a systematic framework for learning these relationships from historical data and generating reliable price estimates.

In this laboratory exercise, you will apply machine learning techniques to develop a predictive model for car prices. The dataset a variety of different vehicle specifications. You will train, evaluate, and compare regressor models to determine their ability to predict prices effectively.

The "SVR.py" file has the code that implements the instructions listed below.

Lab Instructions

Requirements:

- Python version: 3.8
- Libraries: pandas, numpy, sklearn, pathlib.

1. **Load the Dataset**
    - Load the "car_prices.csv" file from your directory and examine its contents.
    - How many samples are in this dataset? What are the input features and target variables for this regression problem?
2. **Split the Dataset**
    - Divide the dataset into training and test sets: 70% for training, 15% validation and 15% for testing.
    - Comment on the number of samples in the training and test sets.
3. **Select and Configure the Algorithm**
    - Use the Support Vector Regressor (SVR) for this regression problem.
4. **Hyperparameter Exploration**
    - Train the SVR with using the combinations of hyperparameters provided in Table 2.
    - For each combination, record the training and validation errors. The root mean-squared error (RMSE) is to be used as the error metric. Comment on the model behavior and indicate if a given set of hyperparameters results in overfitting, underfitting, or good generalization (**NB:** A good performing model for this problem should have an RMSE of 80,000 or less.)

- Complete this task using the template shown in Table 2.

Table 2: Performance of the SVR using different combinations of hyperparameters

Hyperparameters	**Training error (RMSE)**	**Val. error (RMSE)**	**Comments (overfitting/underfitting/good generalization)**
kernel='linear', C=0.1, epsilon=0.01, tol=1e-3, max_iter=-1			
kernel='linear', C=0.1, epsilon=0.05, tol=1e-3, max_iter=-1			
kernel='linear', C=0.1, epsilon=0.1, tol=1e-4, max_iter=-1			
kernel='linear', C=0.5, epsilon=0.01, tol=1e-3, max_iter=-1			
kernel='linear', C=0.5, epsilon=0.05, tol=1e-4, max_iter=-1			
kernel='linear', C=0.5, epsilon=0.1, tol=1e-4, max_iter=1000			
kernel='linear', C=1.0, epsilon=0.01, tol=1e-3, max_iter=-1			
kernel='linear', C=1.0, epsilon=0.05, tol=1e-4, max_iter=-1			
kernel='linear', C=1.0, epsilon=0.1, tol=1e-5, max_iter=5000			
kernel='linear', C=2.0, epsilon=0.01, tol=1e-3, max_iter=-1			
kernel='linear', C=2.0, epsilon=0.05, tol=1e-4, max_iter=-1			
kernel='linear', C=2.0, epsilon=0.1, tol=1e-5, max_iter=5000			
kernel='sigmoid', C=0.1, epsilon=0.01, tol=1e-3, max_iter=-1			
kernel='sigmoid', C=0.1, epsilon=0.05, tol=1e-3, max_iter=-1			

kernel='sigmoid', C=0.1, epsilon=0.1, tol=1e-4, max_iter=-1			
kernel='sigmoid', C=0.5, epsilon=0.01, tol=1e-3, max_iter=-1			
kernel='sigmoid', C=0.5, epsilon=0.05, tol=1e-4, max_iter=-1			
kernel='sigmoid', C=0.5, epsilon=0.1, tol=1e-4, max_iter=1000			
kernel='sigmoid', C=1.0, epsilon=0.01, tol=1e-3, max_iter=-1			
kernel='sigmoid', C=1.0, epsilon=0.05, tol=1e-4, max_iter=-1			
kernel='sigmoid', C=1.0, epsilon=0.1, tol=1e-5, max_iter=5000			
kernel='sigmoid', C=2.0, epsilon=0.01, tol=1e-3, max_iter=-1			
kernel='sigmoid', C=2.0, epsilon=0.05, tol=1e-4, max_iter=-1			
kernel='sigmoid', C=2.0, epsilon=0.1, tol=1e-5, max_iter=5000			
kernel='poly', C=0.1, epsilon=0.01, tol=1e-3, max_iter=-1			
kernel='poly', C=0.1, epsilon=0.05, tol=1e-3, max_iter=-1			
kernel='poly', C=0.1, epsilon=0.1, tol=1e-4, max_iter=-1			
kernel='poly', C=0.5, epsilon=0.01, tol=1e-3, max_iter=-1			
kernel='poly', C=0.5, epsilon=0.05, tol=1e-4, max_iter=-1			
kernel='poly', C=0.5, epsilon=0.1, tol=1e-4, max_iter=1000			
kernel='poly', C=1.0, epsilon=0.01, tol=1e-3, max_iter=-1			
kernel='poly', C=1.0, epsilon=0.05, tol=1e-4, max_iter=-1			
kernel='poly', C=1.0, epsilon=0.1, tol=1e-5, max_iter=5000			
kernel='poly', C=2.0, epsilon=0.01, tol=1e-3, max_iter=-1			

kernel='poly', C=2.0, epsilon=0.05, tol=1e-4, max_iter=-1			
kernel='poly', C=2.0, epsilon=0.1, tol=1e-5, max_iter=5000			
kernel='poly', C=5.0, epsilon=0.01, tol=1e-4, max_iter=-1			
kernel='rbf', C=0.1, coef0=0.0, tol=1e-3, max_iter=-1			
kernel='rbf', C=0.1, coef0=0.1, tol=1e-3, max_iter=-1			
kernel='rbf', C=0.1, coef0=0.0, tol=1e-4, max_iter=1000			
kernel='rbf', C=0.5, coef0=0.0, tol=1e-3, max_iter=-1			
kernel='rbf', C=0.5, coef0=0.1, tol=1e-4, max_iter=-1			
kernel='rbf', C=0.5, coef0=0.0, tol=1e-5, max_iter=5000			
kernel='rbf', C=1.0, coef0=0.0, tol=1e-3, max_iter=-1			
kernel='rbf', C=1.0, coef0=0.1, tol=1e-4, max_iter=-1			
kernel='rbf', C=1.0, coef0=0.0, tol=1e-5, max_iter=10000			
kernel='rbf', C=2.0, coef0=0.0, tol=1e-3, max_iter=-1			
kernel='rbf', C=2.0, coef0=0.1, tol=1e-4, max_iter=-1			
kernel='rbf', C=5.0, coef0=0.0, tol=1e-5, max_iter=5000			
kernel='rbf', C=5.0, coef0=0.1, tol=1e-5, max_iter=10000			

5. **Cross-Validation Analysis**
    - Repeat steps 4 and 5 using cross-validation techniques:
        - K-fold for 5, 10 and 15 folds (3 tables)
        - Repeated K-fold for 5, 10 and 15 folds with 5 and 10 repetitions each (6 tables)
6. **Hyperparameter selection**
    - Identify the best-performing models based on your hyperparameter investigation for the random split, K-fold and repeated K-fold cross-validation.
7. **Compare and Conclude**
    - Evaluate the performance of the model on the test data. Here, the training and validation data are combined into one general dataset for training the regressor.

- Obtain the RMSE AND R^2 coefficient using the selected set of hyperparameters based on the 3 techniques. If the same set of hyperparameters produces the best performance in all 3 cases, you may report the result once for all.
- Discuss the results.

8. "StandardScaler" is used to preprocess the numerical features and target variable to enhance the learning ability of the SVR. Using steps 4 – 7, you may investigate the performance of the model when other scalers such as "MinMaxScaler", "MaxAbsScaler" are used, for some insight. Although it is not a mandatory task for Lab 1, this will be explored further in Lab 2.

List of deliverables:

- Video (webm, mp4 (most preferred), mov)
  - Maximum of 10 minutes in length
  - Not larger than 300 MB
  - Your webcam must be on for this recording. This is mandatory.

- Lab report (pdf file)
  Note: Each file should be named using this template: "student number_reg_lab1.file type"

- **Present all results and answers in a single PDF file. The video recordings for the two tasks could be merged or separated so long as the 10-minute limit is not exceeded.**
- **Submit your work in a folder named using this template: *"student number_lab1.zip"***